

A Hybrid XGBoost–CNN-LSTM Two-Stage Framework for Real-Time Intrusion Detection in Smart City Networks

Divyam Tank
Departement of Computer Engineering
Marwadi University
Rajkot, India
divyamtank01@gmail.com

Hetu Prajapati
Departement of Computer Engineering
Marwadi University
Rajkot, India
hetuprajapati@gmail.com

Dr. Anjali Diwan
Departement of Computer Engineering
Marwadi University
Rajkot, India
anjali.diwan@marwadieducation.edu.in

Abstract— *The growing volume and complexity of threats targeting large-scale networked infrastructure has placed conventional single-model intrusion detection architectures under increasing strain. Systems that treat binary attack identification and fine-grained attack classification as a unified problem impose unnecessary computational overhead on the detection stage while often performing poorly on rare attack categories — a trade-off that becomes untenable in environments where millions of flows must be processed per second. This paper proposes a two-stage hybrid detection framework in which Extreme Gradient Boosting (XGBoost) operates as a high-throughput binary gate, and a Convolutional Neural Network combined with Long Short-Term Memory (CNN-LSTM) subsequently identifies the specific attack category among twenty distinct classes. The framework is evaluated on NF-UQ-NIDS-v2, a large-scale unified NetFlow benchmark of 75,987,976 records consolidated from four major network environments. XGBoost achieves 98.54% binary detection accuracy with an AUC-ROC of 99.87%, while CNN-LSTM reaches 99.46% multiclass accuracy and a weighted F1-score of 99.25% in end-to-end pipeline conditions. The pipeline sustains a throughput of 77,563 flows per second over 22,796,391 test records, with the staged design reducing CNN-LSTM inference load by 33% through selective routing of flagged traffic only. Per-class analysis reveals perfect classification on high-frequency attack categories including DDoS, DoS, scanning, XSS, injection, and password, while rare categories such as Worms and Shellcode remain limited by data insufficiency rather than architectural constraints. These results demonstrate that architectural separation of detection and classification tasks yields measurable gains in both computational efficiency and analytical clarity compared to monolithic approaches.*

I. INTRODUCTION

The Smart City is no longer a dream it has become a growing reality. Globally, cities are integrating transport infrastructure, energy, healthcare systems, and utility systems into one intercommunicado digital system. This connectivity offers tremendous operational advantages, but as well generates something much more perilous: a tremendous, heterogeneous attack

surface, which-adversaries are now willing and capable of exploiting more than ever-present. In contrast to conventional enterprise networks, smart city infrastructure is extremely important, an effective intrusion does not just leak information, it can easily shut down traffic lights, or can cut power supply, or can even silence emergency response systems. Securing these networks is not a choice, but a civic duty [1].

The Network Intrusion Detection System is the center of this protection effort. NIDS used signatures, libraries of known attack patterns, compared to observed traffic, over decades. These systems proved to be effective in times when the threat landscape was predictable but they are essentially blind to anything that they have not encountered before [2]. With increasingly advanced and new forms of attacks, the research community resorted to machine learning. The initial ML models, including Decision Trees, Naive Bayes, Support Vector Machines, shows that statistical models can be trained to learn the form of malicious traffic using labelled data, and extrapolate to novel signatures. However, these techniques have a blind spot, that they considered each network flow as a single observation, entirely ignoring the time story that traverses a real attack. A port scan is not a packet but it is a series. A DDoS campaign is also not a single flow but it is a process that takes time. The models that fail to observe this structure will never get the whole picture [2].

Deep learning offers a path for such problems. Long Short-Term Memory networks bring the ability to model sequences of flows, learning the temporal signatures of multi-stage attacks. Hybrid CNN-LSTM architectures adds spatial feature extraction on top of temporal modelling, and results on earlier benchmark datasets were

compelling [3]. Two-stage pipelines went even a step further and instead of using a single model to answer two entirely different questions, they split binary detection and multiclass classification, which allows each component to do what it does best [4]. These were significant developments. However, there was a more muffled issue in the background: the datasets on which these models were trained and tested were all dissimilar. Various features, various formats, various environments - almost impossible to tell whether a model that had been successful on one dataset would be successful on another, or in a real network. This was directly addressed by Sarhan et al. [13], who merged four large benchmark datasets into one standardised 43-feature NetFlow representation, NF-UQ-NIDS-v2, and showed that an Extra Trees classifier could achieve 97.90% accuracy on its 20 attack classes. This was a big move, but it still brought a new question that could a more advanced staged architecture do much better and could it do quickly enough to have a real smart city network?

This is the question that this paper answers. We suggest a two-stage pipeline with a hybrid approach where XGBoost will conduct an efficient binary screening of all incoming flows and then a CNN-LSTM classifier subsequently identifies the specific attack category among 20 classes for flagged traffic only. Separating these tasks and routing benign traffic away before it enters the deep learning stage, the pipeline has a binary accuracy of 98.54, a multiclass accuracy of 99.46, and an end-to-end throughput of 77,563 flows per second, comfortably meeting the real-time requirements of smart city network backbones.

II. LITERATURE REVIEW

To understand the work on NIDS, we shall follow the path that led to the present state of intrusion detection system. The initial work on NIDS is characterized by anomaly thresholds and rule-based systems that worked well against known threats, but poorly against novel threats. This was fundamentally altered by the transition to machine learning, scientists discovered that models could learn the statistical print of malicious traffic, and this was a tool that the domain could generalize on it. However, as Anis et al. [1] report in their extensive survey of deep learning-based NIDS, this development was accompanied by drawbacks such

as class imbalance, heterogeneity of datasets, and the cost of large-scale inference. These were unresolved issues which carried the field through one generation of models to another.

Ennaji et al. [2] introduce another dimension to this. The more ML-based NIDS became capable, the more vulnerable they became. Attackers with knowledge of the detection system can now design network flows that bypass its blind spots, and the more complicated the model is, the more it provides attackers with an opportunity to manipulate it.

The need to go lighter and more deployable was one of the options that arose to this. Researchers realised that a NIDS which cannot execute in real time on the hardware available is not a NIDS, it is merely a research finding. Chen et al. [6] followed this up with a Deep Forest architecture with hybrid feature selection, and demonstrated that it could outperform six ML and deep learning baselines on four IoT datasets and can be resource-competitive to run on the edge. Ebrahimi et al. [7] followed a similar pattern to IoT settings, with a 1D-CNN lightweight and SHAP-based feature selection to achieve 99.5% accuracy on TON-IoT and with inbuilt explainability that allowed the model to be audited in operational conditions. Azar et al. [8] examined this efficiency question in the setting of satellite-terrestrial networks, which used Random Forest feature selection with LSTM and GRU backends, and discovered that performance differed dramatically across environments it was a reminder that efficiency without generalisability is half a solution.

Another thread dealt with the data issue. When models trained in one environment fail in a different environment, the solution was to fill the gaps with more complex and more diverse training data or synthetic data. GANs were employed by Park et al. [9] to generate samples of minority classes, and they actively enhanced the ability to detect minority attack types that were not be provided by standard datasets in sufficient quantities. Hassan et al. [10] took a different approach by considering more than just header-level NetFlow statistics, considering instead payload embeddings, more semantically rich representations that are more resilient to the

obfuscation techniques employed by attackers to go unnoticed. At the hardware end, Chen et al. [11] constructed the RIDE framework based on Recurrent Autoencoders and memristor hardware policies, with 99.9 percent accuracy and inference speeds that are four orders of magnitude faster than standard DNN it was a glimpse of what can be achieved when the detection architecture is designed to the deployment constraint.

With all this, a more architectural understanding emerged about flow that whether it is malicious and if yes then the multiclass question of what attack it is. These are distinct problems that have to be addressed by different models. This was shown by Hariharan et al. [3], who used their Seq2Seq and ConvLSTM hybrid model, which inserts convolutional operations within recurrent cells to learn spatial-temporal dependencies at the same time, achieving 98.90% accuracy on CIC-ToN-IoT. Ahmed et al. [12] demonstrated that gradient boosting which also specifically XGBoost is exceptionally well to the binary detection issue in IoT traffic and that flow duration features are the most effective signal in RT-IoT2022 with flow duration features regularly being the most effective signal.

Hnamte et al. [4] connected these threads to each other in the most direct way, suggesting the LSTM-AE two-stage framework that divided the anomaly detection and classification tasks across CICIDS2017 and CSE-CICIDS2018, reaching up to 99.99% accuracy and showing that two-stage pipelines can actually be more effective than monolithic architectures. The other section was whether this paradigm would be true at a much greater scale on a more diverse and unified benchmark. Sarhan et al. [13] have just that benchmark on NF-UQ-NIDS-v2 dataset, which consolidated 75,987,976 flows in 20 attack classes into a standardised 43-feature NetFlow representation and reported that an Extra Trees baseline achieved 97.90% accuracy, but the problem of rare attack classes and real-time throughput was still present. This unified benchmark concept was previously proven by the same group [5] with the value of NetFlow standardisation across constituent datasets. The current work uses the two-stage architecture of [4] and adapts it to the scale and diversity of [13],

substituting the autoencoder stage with a gradient-boosted binary gate to optimise smart city edge throughput, and the classification stage with a CNN-LSTM for multi-class attack classification at 77,563 flows per second.

III. METHODOLOGY

The suggested framework addresses the problem of intrusion detection as a pipeline issue and not as a single classification problem. Binary detection and multiclass identification are addressed by different components, each designed to suit the needs of its sub-problem. Raw network data is transformed into a labeled output through five stages: dataset preparation, feature engineering, Stage 1 binary screening with XGBoost, Stage 2 attack classification with CNN-LSTM, and inference execution.

A. Dataset Preparation

NF-UQ-NIDS-v2 [5] is the benchmark of evaluation in this work. The reason why it is so well-suited is its construction methodology is instead of gathering fresh traffic, Sarhan et al. reconstructed NetFlow records from the raw packet captures of four existing datasets (UNSW-NB15, BoT-IoT, ToN-IoT, CSE-CIC-IDS2018) using the nProbe tool, producing a single unified 43-feature representation [13]. The outcome is 75,987,976 labeled flow records across 20 distinct attack categories and benign traffic of the largest and most diverse publicly available NIDS benchmark.

The class distribution is a genuine challenge rather than a statistical nuisance. Benign traffic accounts for 33.12% of records (25,165,295 flows) and attacks for the remaining 66.88% (50,822,681 flows). Within the attack segment the spread is extreme as DDoS contributes more than 21 million flows while Worms appears only 164 times. This is the reality of network traffic and it must be explicitly handled at each stage of the pipeline. A stratified 70/30 train-test split preserves these proportions across both partitions, yielding a training corpus of approximately 53 million records and a test set of 22,796,391 flows.

B. Feature Engineering

Reducing 43 features to a generalisable subset requires eliminating anything that binds the model

to a particular network topology. Source and destination IP addresses and ephemeral source ports are dropped a model that learns to associate specific IP ranges with attack behaviour will fail immediately when exposed to a different network [14]. The 21 retained features are selected for their coverage of security-relevant events across varying traffic conditions. Table I summarises the retained feature set by category.

Table I. Retained Feature Set

Category	Features
Flow Duration	FLOW_DURATION_MILLISEC ONDS, DURATION_IN, DURATION_OUT
Volume	IN_BYTES, OUT_BYTES, IN_PKTS, OUT_PKTS
TCP Flags	TCP_FLAGS, CLIENT_TCP_FLAGS, SERVER_TCP_FLAGS
Application Layer	L7_PROTO, L4_DST_PORT, PROTOCOL, ICMP_TYPE, DNS_QUERY_ID, DNS_QUERY_TYPE, FTP_COMMAND_RET_CODE
Throughput & Packet Size	SRC_TO_DST_AVG_THROUGH PUT, DST_TO_SRC_AVG_THROUGH PUT, LONGEST_FLOW_PKT, SHORTEST_FLOW_PKT

Feature normalisation follows directly. As Umar et al. [14] demonstrated, normalisation strategy produces a quantifiable impact on detection accuracy and false alarm rates is an effect that is easy to overlook and costly to neglect in practice. Each feature j of flow record i is standardised as:

$$x'_{ij} = \frac{x_{ij} - \mu_j}{\sigma_j} \quad (1)$$

where μ_j and σ_j are the mean and standard deviation computed on the first training chunk only. These parameters are fixed and applied unchanged to all subsequent processing stages including inference, ensuring that no test-set information leaks into the scaling parameters.

C. Stage 1 -XGBoost Binary Classifier.

The binary gate must be fast, reliable, and deployable on hardware that bears no resemblance

to a data centre. XGBoost fits this description precisely as the gradient-boosted tree ensemble is interpretable, computationally lean at inference, and consistently strong on tabular flow data, as Ahmed et al. [12] confirmed with accuracy exceeding 99.5% on IoT NIDS benchmarks. The model minimises the regularised objective:

$$\mathcal{L} = \sum_{i=1}^n \ell(y_i, \hat{y}_i) + \sum_{k=1}^K \Omega(f_k) \quad (2)$$

where ℓ is binary cross-entropy, f_k are the constituent regression trees, and $\Omega(f_k)$ is a regularisation term controlling tree complexity.

Training 53 million records in a single pass is not feasible on standard hardware. The booster is therefore trained incrementally on one million records per chunk, ten boosting rounds per chunk is an approach whose mathematical outcome is identical to batch training while the memory footprint remains bounded. This matters directly for deployment at smart city edge nodes where RAM is constrained and loading multi-gigabyte datasets in a single pass is not viable.

The class imbalance between benign and attack flows is managed by setting w_{pos} as:

$$w_{\text{pos}} = \frac{N_{\text{benign}}}{N_{\text{attack}}} = \frac{25,165,295}{50,822,681} \approx 0.495 \quad (3)$$

This value is passed to the `scale_pos_weight` parameter, which modifies gradient updates during tree construction so the model does not collapse toward the majority class. Remaining hyperparameters are: `max_depth=6`, `learning_rate=0.1`, `subsample=0.8`, `colsample_bytree=0.8`, with histogram-based tree construction (`tree_method=hist`) and GPU acceleration (`device=cuda`). At inference, any flow whose predicted attack probability falls below $\tau=0.5$ is classified as benign and exits the pipeline immediately. All flows above this threshold proceed to Stage 2.

D. Stage 2 CNN-LSTM Multiclass Classifier.

A single flow rarely tells the complete story of an attack. Port scans unfold across dozens of successive connection attempts; DDoS floods sustain characteristic patterns over hundreds of flows. The CNN-LSTM is built around this

observation that it classifies sequences of flows rather than individual records, seeking the temporal fingerprints that single-record models cannot identify by their nature.

Attack flows flagged by Stage 1 are grouped into non-overlapping windows of $T=20T = 20$ consecutive records. During training, sequences are constructed within attack categories to ensure clean labels is mixing attack types within a single window would introduce irresolvable ambiguity. This grouping is not available at inference, and the labelling noise introduced by mixed-type buffers is a known cost of realistic deployment that is measured explicitly in Section IV.

The architecture processes each sequence window S through two convolutional blocks for local feature interaction — 64 filters of width 3 with batch normalisation, followed by 128 filters of the same width with batch normalisation, max pooling (pool_size=2), and dropout (rate=0.3). The convolutional output feeds into two stacked LSTM layers (128 units with return_sequences=True, then 64 units), each followed by dropout (rate=0.3), and a 64-unit dense layer with dropout (rate=0.2) before the final classification layer. The model output is:

$$\mathbf{z} = \text{CNN} - \text{LSTM}(\mathbf{S}), p_c = \frac{e^{z_c}}{\sum_{j=1}^{20} e^{z_j}} \quad (4)$$

where p_c is the predicted probability for attack class c across 20 categories.

Rare classes such as Worms (164 training records) would be practically invisible under standard cross-entropy loss. This is addressed through inverse-frequency class weighting, where each sample's loss contribution is scaled as:

$$w_c = \frac{N_{\text{total}}}{C \times N_c} \quad (5)$$

where N_{total} is the total number of training attack records, $C=20$ is the number of attack classes, and N_c is the count for class c . The weighted classification loss becomes:

$$\mathcal{L}_{\text{cls}} = - \sum_{i=1}^N w_{y_i} \log(p_{i,y_i}) \quad (6)$$

The rarer the class, the higher its weight would be, Worms receives a weight of 17,787 against DDoS at 0.117. Training uses the Adam optimiser with

early stopping on validation loss (patience=5) and best-checkpoint restoration. The model runs on a JAX backend with TPU acceleration at a batch size of 2,048.

E. End-to-End Inference

At inference time the pipeline processes the full test set in successive chunks. Each chunk is standardised using equation (1), evaluated by XGBoost using equation (2), and divided: flows below threshold τ are immediately routed to the benign output, flows at or above τ are written to a disk-based memory-mapped file. Holding the full attack buffer in RAM would require tens of gigabytes — disk offloading eliminates this constraint without affecting throughput meaningfully.

Once all chunks have been processed, the attack buffer is read back in batches of 40,000 rows, arranged into sequences of length $T=20T = 20$ $T=20$, and passed through CNN-LSTM using equation (4). The final label for each flow is assigned according to the two-stage routing rule:

$$\hat{y}_i = \begin{cases} \text{Benign}, & p_i^{(\text{bin})} < \tau \\ \arg \max_{c \in \{1, \dots, 20\}} p_i, c, & p_i^{(\text{bin})} \geq \tau \end{cases} \quad (7)$$

where $p_i^{(\text{bin})}$ is the XGBoost attack probability for flow i , p_i, c is the CNN-LSTM class probability for category c , and $\tau = 0.5$ is the binary decision threshold. This single expression captures the entire architectural contribution of the paper: XGBoost determines whether a flow is malicious, and CNN-LSTM identifies what kind of attack it is and the two questions are answered sequentially by models suited to each.

IV. RESULT AND ANALYSIS

The evaluation is conducted on a test set of 22,796,391 network flow records drawn from the NF-UQ-NIDS-v2 benchmark. All figures reported in this section reflect end-to-end pipeline performance unless explicitly stated otherwise — that is, XGBoost binary detection followed by CNN-LSTM multiclass classification on the flagged attack subset, processed sequentially as described in the methodology. Standalone model

figures are reported where they provide useful context for understanding the pipeline's behaviour.

A. Stage 1: XGBoost Binary Detection

Over the full 22,796,391-record test set, XGBoost achieves 98.54% accuracy, 99.50% precision, 98.30% recall, and an F1-score of 98.90%. The AUC-ROC of 99.87% and AUC-PR of 99.94% (Fig 1) confirm that performance holds across the complete range of operating thresholds — not just at the default point. The binary confusion matrix is shown in Fig 2.

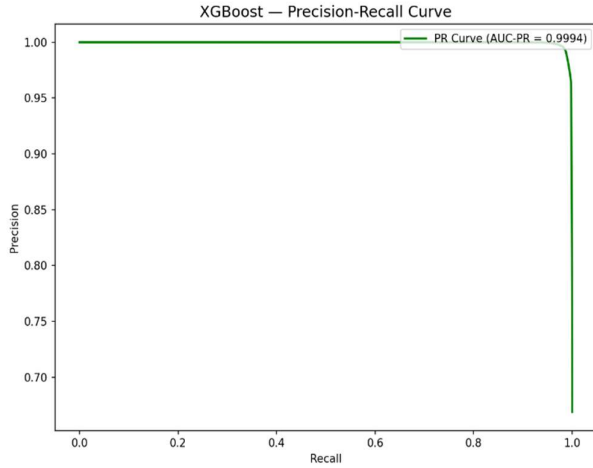


Figure 1. XGBoost- Precision-Recall Curve

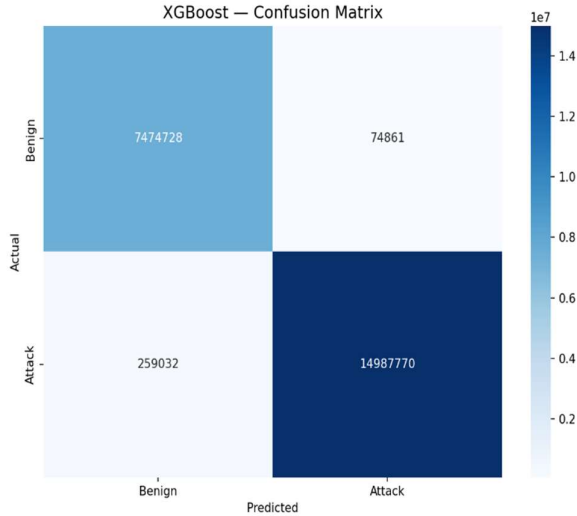


Figure 2. XGBoost Confusion Matrix

Of the test flows, 7,474,728 benign and 14,987,770 attack flows are correctly classified. The 74,861 false positives enter Stage 2 as misrouted benign flows — an error that is visible and partially

recoverable downstream. The 259,032 false negatives are more consequential: they exit the pipeline labelled benign and never reach Stage 2, establishing a hard ceiling of 1.70% on pipeline-level attack miss rate.

Feature importance analysis using the gain criterion places DURATION_IN as the dominant discriminator by a wide margin — gain score 3,241, against 413 for FLOW_DURATION_MILLISECONDS and 132 for L4_DST_PORT. That temporal features outrank volumetric ones reflects a real property of the dataset: DDoS and DoS attacks sustain flow durations that are statistically distinct from benign traffic in ways that byte counts alone do not reliably capture. Ahmed et al. [12] independently observed the same dominance of flow duration features on RT-IoT2022, lending some confidence that this is not dataset-specific. At the bottom of the ranking, DNS_QUERY_ID and DNS_QUERY_TYPE contribute negligible gain — both can be safely deprioritised in lightweight edge deployments where feature extraction cost matters.

B. CNN-LSTM Multiclass Classification

In standalone evaluation with pre-grouped homogeneous attack sequences, CNN-LSTM reaches 99.97% accuracy, 99.97% weighted F1-score, and AUC-ROC of 1.000 across all 20 classes — establishing what the model achieves under ideal conditions. In the end-to-end pipeline, where flagged flows arrive without category grouping and sequence windows routinely span multiple attack types, weighted accuracy settles at 99.46% and weighted F1-score at 99.25%. The gap between these two figures is not a failure — it is the honest cost of realistic operating conditions, where mixed-label sequence windows introduce ambiguity the temporal model must resolve without ground truth context. Reporting both figures matters: standalone alone would overstate pipeline performance, pipeline alone would obscure where the degradation originates.

C. Per-Class Analysis

Table II presents the full per-class classification report from the pipeline evaluation over 15,062,631 attack-flagged records.

Table II. Per-Class Classification Report Pipeline Evaluation

Attack Category	Precision	Recall	F1	Support
DDoS	1.00	1.00	1.00	6,522,161
DoS	1.00	1.00	1.00	5,348,527
Scanning	0.99	1.00	1.00	1,110,109
XSS	1.00	1.00	1.00	735,377
Password	1.00	1.00	1.00	345,515
Injection	0.99	1.00	1.00	203,964
Reconnaissance	0.92	1.00	0.96	610,046
Bot	1.00	1.00	1.00	42,929
Brute Force	0.99	1.00	1.00	37,097
Backdoor	0.96	0.98	0.97	5,602
Exploits	0.95	0.96	0.96	8,830
Fuzzers	0.93	0.94	0.94	6,082
Generic	0.98	0.98	0.98	4,821
Ransomware	0.81	0.80	0.80	815
MITM	0.62	0.86	0.72	1,796
Analysis	0.62	0.80	0.70	586
Theft	0.76	0.74	0.75	575
Infiltration	0.15	0.79	0.25	2,655
Shellcode	0.53	0.40	0.46	237
Worms	0.00	0.00	0.00	46
Weighted Avg	0.99	0.99	0.99	15,062,631
Macro Avg	0.77	0.82	0.78	15,062,631

The high-volume categories perform without fault. DDoS, DoS, scanning, XSS, password, and injection all reach F1-scores of 1.00, covering the categories a deployed system will encounter most frequently. Bot, Brute Force, and Backdoor follow closely. Together these represent the vast majority of observed attack traffic, and on this criterion the pipeline is reliable.

The rare and structurally ambiguous categories deserve direct discussion rather than a footnote. Infiltration achieves recall of 0.79 but precision of only 0.15 — the model finds most actual Infiltration flows but generates a heavy volume of false positives by pulling in flows from Generic and Reconnaissance. The overlap between these categories in NetFlow feature space is a dataset-level characteristic rather than a modelling failure, amplified in the pipeline by mixed-type sequence windows that present the temporal model with ambiguous input. Shellcode (F1: 0.46, 237 records) confuses primarily with Reconnaissance. Worms at 46 test records is statistically uninterpretable — the training data is simply insufficient for this category regardless of architecture. Park et al. [9] demonstrated that GAN-based synthesis of minority class samples can materially improve detection of exactly these rare categories, and their integration as a preprocessing step is the most direct path forward.

The macro-average F1-score of 0.78 is the most honest single number this section produces. The gap between it and the weighted average of 0.99 is entirely explained by the rare categories above. Both figures should be read together — weighted describes behaviour on traffic the system will actually see, macro describes where the remaining work is.

D. End-to-End Pipeline Results

The complete pipeline evaluated on 22,796,391 test flows achieves 98.54% binary detection accuracy at Stage 1 and 99.46% multiclass classification accuracy on the attack-flagged subset at Stage 2. These are not independent figures — they are connected by the routing decision made at the binary gate. The 259,032 flows that Stage 1 misses as false negatives never reach Stage 2 and are permanently lost to the pipeline. The 74,861 false positives that Stage 1 incorrectly flags do

reach Stage 2, where they are assigned an attack label — a compounded error that the per-class report captures in the Benign row's zero F1-score.

Taken together, the weighted-average precision of 99.06%, recall of 99.46%, and F1-score of 99.25% over all 15,062,631 attack-flagged records represent the system's actual operational performance — not a best-case laboratory result. The staged design reduces CNN-LSTM invocations to 66.9% of total flows, since 33.1% are resolved as benign at Stage 1 without ever engaging the deep learning component. Against a hypothetical single-model CNN-LSTM applied uniformly to all flows, this yields approximately 33% reduction in deep learning inference load while maintaining equivalent classification accuracy on the attack subset.

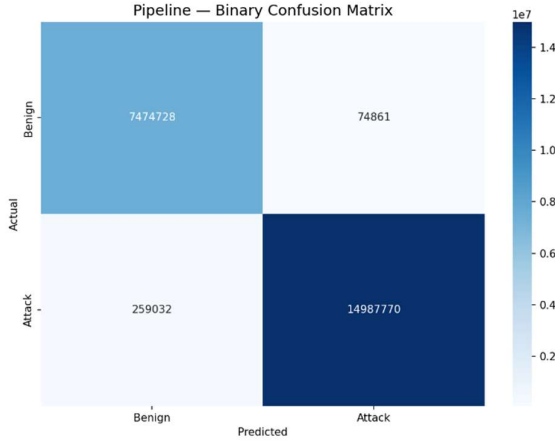


Figure 3. Pipeline Binary Confusion Matrix

E. Comparative Analysis

Table II places the proposed framework against published results on related benchmarks. On the same NF-UQ-NIDS-v2 dataset, the full pipeline at 99.46% accuracy and 99.25% F1 clears both baselines reported by Sarhan et al. [13] — Extra Trees at 97.90% and Random Forest at 97.80% — by a meaningful margin. Across datasets, the CNN-LSTM stage is broadly comparable to Hnamte et al. [4] at 99.20% and sits above Hariharan et al.'s [3] 98.90%, though dataset differences limit direct interpretation in both cases. Ahmed et al.'s [12] XGBoost result of 99.55% on RT-IoT2022 aligns closely with Stage 1 behaviour, and the shared dominance of flow duration features across both studies adds confidence that the binary gate generalises beyond this benchmark. As Anis et al.

[1] observe, binary detection accuracy has broadly converged — the remaining gap is rare-class multiclass performance, and data augmentation [9] is the more productive next step there than architectural changes.

Table III. Comparative Results

Study	Dataset	Accuracy (%)	F1 (%)	AUC-ROC (%)
Sarhan et al. — Extra Trees [13]	NF-UQ-NIDS-v2	97.90	97.87	98.30
Sarhan et al. — Random Forest [13]	NF-UQ-NIDS-v2	97.80	97.77	98.10
Hnamte et al. — LSTM-AE [4]	CICIDS2017	99.20	99.10	N/A
Hariharan et al. — ConvLSTM [3]	CIC-ToN-IoT	98.90	98.80	N/A
Ahmed et al. — XGBoost [12]	RT-IoT2022	99.55	N/A	N/A
Proposed — XGBoost (Stage 1)	NF-UQ-NIDS-v2	98.54	98.90	99.87
Proposed — CNN-LSTM (Stage 2)	NF-UQ-NIDS-v2	99.46	99.25	100.00
Proposed — Full Pipeline	NF-UQ-NIDS-v2	99.46	99.25	99.87

F. Throughput and Efficiency

The pipeline processes all 22,796,391 test flows in 293.91 seconds, sustaining 77,563 flows per second end-to-end. XGBoost accounts for 269 seconds at roughly 84,700 flows per second across the full test set. CNN-LSTM processes its 15,246,802 flagged records in 24.94 seconds. The asymmetry is straightforward — XGBoost touches every flow, CNN-LSTM touches only what passes the gate.

Routing 7,549,589 benign flows — 33.1% of the test set — directly to output without engaging CNN-LSTM produces a real reduction in deep learning inference load compared to applying a temporal model uniformly across all traffic. For a smart city backbone generating between 10,000 and 50,000 flows per second, this throughput provides comfortable headroom for real-time operation with capacity to absorb traffic spikes.

V. CONCLUSION

The problem this paper set out to address was not accuracy — existing work had already shown that neural networks can classify known attack categories with high fidelity on standard benchmarks. The problem was architectural. A detection system that applies an expensive temporal model uniformly to all traffic is not designed for the environment it must operate in. Smart city networks generate millions of flows per second across heterogeneous infrastructure, and any framework that cannot keep pace is, as a practical matter, not a detection framework at all. The two-stage pipeline proposed here addresses this directly. XGBoost acts as a fast binary gate, routing 33.1% of traffic — the benign majority — away from the deep learning stage entirely. CNN-LSTM then applies temporal sequence modelling only where it adds the most value: on traffic already identified as likely malicious. The pipeline achieves 98.54% binary detection accuracy with an AUC-ROC of 99.87%, 99.46% multiclass accuracy with a weighted F1-score of 99.25%, and throughput of 77,563 flows per second over 22,796,391 test records. Per-class results are equally revealing: F1-scores reach 1.00 on DDoS, DoS, scanning, XSS, injection, and password attacks, but rare classes such as Worms, Shellcode,

and Infiltration remain limited by data, not architecture. Future work should therefore focus on GAN-based minority synthesis, adversarial hardening, federated training, and online updating for concept drift.

REFERENCES

- [1] F. M. ANIS, M. ALABDULLATIF, S. ALJBLI, AND M. HAMMOUDEH, "A SURVEY ON THE APPLICATIONS OF DEEP LEARNING IN NETWORK INTRUSION DETECTION SYSTEMS TO ENHANCE NETWORK SECURITY," *IEEE ACCESS*, VOL. 13, PP. 185357–185380, 2025.
- [2] S. ENNAJI, F. DE GASPARI, D. HITAJ, A. KBIDI, AND L. V. MANCINI, "ADVERSARIAL CHALLENGES IN NETWORK INTRUSION DETECTION SYSTEMS: RESEARCH INSIGHTS AND FUTURE PROSPECTS," *IEEE ACCESS*, VOL. 13, PP. 148613–148645, 2025.
- [3] S. HARIHARAN, Y. A. JERUSHA, G. SUGANESHWARI, S. P. S. IBRAHIM, U. TUPAKULA, AND V. VARADHARAJAN, "A HYBRID DEEP LEARNING MODEL FOR NETWORK INTRUSION DETECTION SYSTEM USING SEQ2SEQ AND CONVLSTM-SUBNETS," *IEEE ACCESS*, VOL. 13, PP. 30705–30720, 2025.
- [4] V. HNAME, H. NHUNG-NGUYEN, J. HUSSAIN, AND Y. HWA-KIM, "A NOVEL TWO-STAGE DEEP LEARNING MODEL FOR NETWORK INTRUSION DETECTION: LSTM-AE," *IEEE ACCESS*, VOL. 11, PP. 37131–37148, 2023.
- [5] M. SARHAN, S. LAYEGHY, N. MOUSTAFA, AND M. PORTMANN, "NETFLOW DATASETS FOR MACHINE LEARNING-BASED NETWORK INTRUSION DETECTION SYSTEMS," IN *PROC. BDTA/WICON*, 2021, PP. 117–135.
- [6] X. Chen, P. Wang, Y. Yang, and M. Liu, "Resource-Constraint Deep Forest-Based Intrusion Detection Method in Internet of Things for Consumer Electronics," *IEEE Trans. Consumer Electronics*, vol. 70, no. 2, pp. 4976–4987, 2024.
- [7] F. Ebrahimi, R. Javidan, R. Akbari, and Y. Hosseini, "Intrusion Detection in the Internet of Things Using Convolutional Neural Networks: An Explainable AI Approach," *Cybersecurity*, vol. 8, p. 66, 2025.
- [8] A. T. Azar, E. Shehab, A. M. Mattar, I. A. Hameed, and S. A. Elsaid, "Deep Learning Based Hybrid Intrusion Detection Systems to Protect Satellite Networks," *J. Network and Systems Management*, vol. 31, 2023.
- [9] C. Park, J. Lee, Y. Kim, J.-G. Park, H. Kim, and D. Hong, "An Enhanced AI-Based Network Intrusion Detection System Using Generative Adversarial Networks," *IEEE Internet of Things Journal*, vol. 10, no. 3, pp. 2330–2342, 2023.
- [10] M. Hassan, M. E. Haque, M. E. Tozal, V. Raghavan, and R. Agrawal, "Intrusion Detection Using Payload Embeddings," *IEEE Access*, vol. 10, pp. 4015–4027, 2022.

[11]J. Chen, G. Adam, L. Zhang, N. D. Bastian, J. Riem, and T. Lan, "RIDE: Real-time Intrusion Detection via Explainable Machine Learning Implemented in a Memristor Hardware Architecture," in *Proc. IEEE INFOCOM*, 2023.

[12]M. A. O. Ahmed, Y. AbdelSatar, R. Alotaibi, and O. Reyad, "Enhancing Internet of Things Security Using Performance Gradient Boosting for NIDS," *Alexandria Engineering Journal*, vol. 116, pp. 472–482, 2025.[13]M. Sarhan, S. Layeghy, and M. Portmann, "Towards a Standard Feature Set for Network Intrusion Detection System Datasets," *Mobile Networks and Applications*, vol. 27, pp. 357–370, 2022.

[13]M. Sarhan, S. Layeghy, and M. Portmann, "Towards a Standard Feature Set for Network Intrusion Detection System Datasets," *Mobile Networks and Applications*, vol. 27, pp. 357–370, 2022.

[14]M. A. Umar, Z. Chen, K. Shuaib, and Y. Liu, "Effects of Feature Selection and Normalization on Network Intrusion Detection," *Data Science and Management*, vol. 8, pp. 23–39, 2025.